StabCert: translation validation for stabilizer channels

Frédéric Kramarczyk
Independent researcher
fkra62@gmail.com

Abstract

Quantum routing compilers rewrite circuits to satisfy hardware connectivity constraints, but the resulting circuits are seldom verified against the channel they are meant to implement. We present a decision procedure for the equality of stabilizer channels — Clifford unitaries with stabilizer ancilla preparation and partial trace — restricted to a specified input subspace.

Our procedure compares the canonical signed signatures of code-Choi states, relative to a support determined by the specification, and is both sound and complete: two such channels agree on the code subspace if and only if their signatures coincide. It runs in polynomial time, avoids dense matrix representations and basis state enumeration, and its verdict is invariant under any change of Stinespring dilation.

We implement this in StabCert, a tool that treats the compiler as an untrusted black box and certifies routed circuits produced by Qiskit SABRE and pytket against targets reconstructed from a specification rather than from a reference circuit. On the families studied, our measurements identify generator density — not circuit width alone — as a key factor in verification cost.

1 Introduction

Quantum programs written for hardware must be routed: a compiler inserts SWAP gates and relabels qubits so that every two-qubit operation acts on an edge of the device coupling map. This transformation is substantial, and it is performed by heuristic search procedures that are not themselves verified. Qiskit's SABRE and pytket's routing passes are widely deployed and largely trusted on the strength of testing.

Existing verification approaches address adjacent problems.

Compiler certification — proving the transformation itself, once and for all — applies only where the compiler's source is available and under the verifier's control. SABRE and pytket are neither.

Equivalence checking, as implemented for example in QCEC [2], compares a compiled circuit against a reference circuit. Like our approach it relies on canonical representations; the distinction is not canonicity but the object represented and the input required. Section 8 states it precisely, together with the capabilities and limits of the engines involved.

Deductive verification proves properties of programs written in a dedicated language, reaching algorithms as complex as Shor's order finding, but takes its input as a program rather than as an artifact produced elsewhere.

None of these answers the question a routing pass raises in practice: given a circuit produced by an untrusted compiler, does it implement the channel the specification calls for? The distinction matters because the reference circuit is itself an artifact. Comparing against it certifies faithfulness to a possibly wrong starting point.

Contribution. We restrict attention to stabilizer channels, as defined in Section 2, and show that within this class the question is decidable in polynomial time. Specifically:

- We define the code-Choi state, a mixed stabilizer state — a state stabilized by a signed subgroup — encoding a channel's action on a specified input subspace, and prove that equality of its canonical signed signature is necessary and sufficient for equality of the channels on that subspace (Section 3).
- We show the verdict is invariant under any change of Stinespring dilation, so that circuits differing only in ancilla convention are correctly accepted (Section 3.4).
- We implement the procedure in StabCert and certify routed circuits produced by SABRE and pytket, with adversarial campaigns reporting no false accepts and no false rejects (Sections 5–6).
- We measure verification cost and separate the contribution of circuit width from that of generator density (Section 7).

The property is proved of the decision procedure; the adversarial campaigns of Section 6 test the implementation that realises it, and no finite corpus could establish the former.

Scope. The class is narrow by design. Gottesman–Knill makes stabilizer circuits classically simulable, which is what permits an exact decision procedure; it also means our results say nothing about non-Clifford gates, noise, or timing. Section 4 states the boundary precisely.

2 Preliminaries

We assume familiarity with the stabilizer formalism and fix notation.

An n-qubit **stabilizer group** is an abelian subgroup of the Pauli group not containing $-I$. It is represented by a **tableau**: for each generator, a binary vector $(x \mid z) \in \mathbb{F}_2^{2n}$ together with a sign. Group operations correspond to $\mathbb{F}_2$ row operations on the binary parts, with signs carried along. The word *tableau* is reserved for this representation throughout.

A **Clifford unitary** maps Pauli operators to Pauli operators under conjugation, and is fully specified by the images of $2n$ generators. A **stabilizer channel** is a Clifford unitary composed with stabilizer ancilla preparation and partial trace over a discarded register.

By the **Gottesman–Knill theorem** [6, 1], stabilizer circuits admit efficient classical simulation: a tableau of $2n$ signed generators determines the state, and each gate updates it in polynomial time. This is what makes the decision procedure of Section 3 possible, and it also bounds it — everything below is confined to the stabilizer fragment.

2.1 Registers and symbols

symbol	object
R	purifying reference for the message, $\|M\|$ wires
M	message, channel input
X	accessible partition, channel output
C	inaccessible complement, traced
$O \subseteq X$	requested output, with $\|O\| = \|M\|$
Λ, Λ'	stabilizer channels under comparison
N, N^c	the channel from M to X, and its complement to C
S	stabilizer group of the source Choi state
S_X	subgroup of S surviving the partial trace onto X
τ_X	marginal $N(I/d)$ on X, with $d = 2^{\|M\|}$
Π	projector onto $\mathrm{supp}(\tau_X)$
k	logical qubits, $k = \|X\| - \|S_X\|$
E	support encoder (Definition 1)
$\mathrm{Ref}(X)$	reference register of the code subspace, $\|X\|$ wires
$\mathrm{sig}(\cdot)$	canonical signed signature (Definition 3)

R and $\mathrm{Ref}(X)$ are distinct registers of different widths and are never interchanged.

3 Deciding equality of stabilizer channels

3.1 Setting

Fix a channel specification P, consisting of an input register M, a set of ancilla qubits with initial stabilizers, a source Clifford U, and an accessible partition X.

Let S be the stabilizer group generated by the specification's input generators conjugated by U, and let

$$S_X = \{S|_X : S \in \mathcal{S},\ \mathrm{supp}(S) \subseteq X\}.$$

be the subgroup surviving the partial trace onto X. Write $\Pi = \prod_i (I + S_i)/2$ for the projector onto the joint $+1$ eigenspace of S_X, and $k = |X| - |S_X|$ for the number of logical qubits. The target subspace is $\mathrm{supp}(\tau_X)$ with $\tau_X = \Pi/2^k$.

Hypothesis 1 (specification-determined support)**.** Π is a function of P alone. No part of the candidate artifact contributes to its construction.

This is what the verdict below is relative to, and it is the hypothesis on which the theorem's usefulness rests: a candidate may claim a support, but that claim is compared against Π, never used to build it. Π is never materialised; it is represented throughout by its signed generators.

3.2 The code-Choi state

Definition 1 (support encoder)**.** Let $\bar{X}_i, \bar{Z}_i$ $(i < k)$ be canonical logical representatives of the code defined by S_X, and D_j, S_j its destabilizer–stabilizer pairs. The support encoder E is the Clifford on $|X|$ qubits determined by

$$EX_iE^\dagger = \bar{X}_i, EZ_iE^\dagger = \bar{Z}_i (i < k)$$
$$EX_{k+j}E^\dagger = D_j, EZ_{k+j}E^\dagger = S_j (j \geq 0)$$

Both families of labels are obtained by canonical $\mathbb{F}_2$ elimination, so E is determined by P and involves no gauge choice. E maps $|\psi\rangle_k \otimes$
$lvert0$
$rangle^{|X|-k}$ into the code space, and carries the signs on the stabilizer tail.

The problem format requires $O \subseteq X$ with $|O| = |M|$, enforced at problem construction; this is the condition under which the two sides of the comparison below are of the same type.

Definition 2 (code-Choi state)**.** Let $\mathrm{Ref}(X)$ be a reference register of $|X|$ qubits. Prepare k Bell pairs between $\mathrm{Ref}(X)$ and the first k wires of X, with the remaining $|X|-k$ wires of each half in $|0\rangle$. Apply the **complex conjugate** $\overline{E}$ to $\mathrm{Ref}(X)$ and the support encoder E to X. Pass X through the candidate channel Λ. The code-Choi state $J_\Pi(\Lambda)$ is the result reduced onto $O \cup Ref(X)$.

The conjugation on the reference is not a convention of convenience. It is the same fact as the transpose appearing in the inversion of Theorem 1, seen from the other side: $\overline{E}$ on $\mathrm{Ref}(X)$ is what makes $\mathrm{Tr}_{\mathrm{Ref}}[(\sigma^T \otimes I)J_\Pi(\Lambda)]$ return $\widetilde{\Lambda}(\sigma)$ rather than $\widetilde{\Lambda}(\sigma^T)$. Stating either without the other leaves the definition and the proof inconsistent with each other.

Three remarks, each of which a reader will otherwise have to reconstruct.

The reduction registers differ on the two sides. The target is reduced onto $R \cup X$; the candidate onto $O \cup Ref(X)$. They are comparable because $|O| = |M|$, which Definition 1 requires.

Three distinct partial traces occur in the chain, and they must not be conflated. R and C are traced to form τ_X; C is traced to form the reduced target; $X \setminus O$ is traced to form the reduced candidate. Only the last belongs to Definition 2.

The state is mixed, on $|O| + |X|$ wires. It is not a purification and not a pure stabilizer state: tracing $X \setminus O$ leaves a state stabilized by a proper subgroup. What the construction avoids is a product of projectors — it returns an independent generating set directly, so no normalisation scalar propagates through the canonical form.

The full reference register is kept, not truncated to k wires, because the encoder mixes the $|X| - k$ padding wires into the code. After encoding no factor is separable, and no subset can be discarded.

3.3 Canonical form and the decision procedure

Definition 3 (canonical signed signature)**.** For a set of independent signed Pauli generators, put the binary vectors $(x \mid z)$ in reduced row echelon form, performing each row operation simultaneously on the binary vector and on the corresponding Pauli string, so that phases follow the elimination rather than being recomputed. The canonical signed signature $\mathrm{sig}(\cdot)$ is the resulting ordered tuple of signed generators.

Lemma 1. Let the generators be independent and generate a group not containing $-I$. Then the canonical signed signature is a normal form: two such sets generate the same signed subgroup if and only if their signatures are equal.

Proof. RREF over $\mathbb{F}_2$ is unique for a given row space, so the binary parts agree exactly when the subgroups agree as unsigned groups. Because each row operation is applied to the Pauli strings in parallel, the phase attached to each echelon generator is the phase of the unique group element with that binary support — unique precisely because $-I \notin S$, without which the group would contain both P and $-P$ for the same support. Hence equality of signatures is equivalent to equality of the signed subgroups. □

The hypothesis $-I \notin S$ is not incidental: the implementation rejects any specification whose generators fail it, together with commutation and independence, before any comparison is attempted.

Theorem 1 (soundness and completeness)**.** Let Λ, Λ' be stabilizer channels and Π as above. Then

$$sig(J_\Pi(\Lambda)) = \mathrm{sig}(J_\Pi(\Lambda')) \iff \Lambda(\rho) = \Lambda'(\rho) for all \rho with \, \mathrm{supp}(\rho) \subseteq \mathrm{supp}(\tau_X)$$

Proof. ($\Leftarrow$) Suppose the channels agree on the code subspace. The states ρ supported in $\mathrm{supp}(\Pi)$ span the operator space on that subspace, so by linearity the two channels agree on all of it, hence on the joint state of Definition 2. The code-Choi states coincide, and by Lemma 1 their signatures are equal.

($\Rightarrow$) Equal signatures give equal signed stabilizer groups, hence $J_\Pi(\Lambda) = J_\Pi(\Lambda')$ as states. Three steps then recover the channels.

First, E restricts to a unitary isomorphism between $\mathbb{C}^{2^k} \otimes |0\rangle^{rvert X rvert - k}$ and the code subspace. By Definition 1, E maps the stabilizer group of $|0\rangle^{rvert X rvert - k}$ on the last $|X| - k$ wires onto S_X, so it carries that subspace onto the joint $+1$ eigenspace of S_X, which is $\mathrm{supp}(\Pi)$; being Clifford it is unitary, hence a bijection between the two. Every ρ with $\mathrm{supp}(\rho) \subseteq \mathrm{supp}(\tau_X)$ is therefore $E(\sigma \otimes |0\rangle\langle 0|)E^\dagger$ for a unique σ on k qubits.

Second, $J_\Pi(\Lambda)$ is the ordinary Choi state of the induced logical channel $\widetilde{\Lambda} : \sigma \mapsto \Lambda(E(\sigma \otimes |0\rangle\langle 0|)E^\dagger)$. The prepared state of Definition 2 is $(\overline{E} \otimes E)(\Phi_k \otimes |0\rangle\langle 0|)(\overline{E} \otimes E)^\dagger$, whose reduction on X is $\Pi/2^k$ and whose correlations with $\mathrm{Ref}(X)$ are those of Φ_k transported by $\overline{E} \otimes E$. Applying Λ to X and reducing onto $O \cup Ref(X)$ therefore yields $(id \otimes \widetilde{\Lambda})(\Phi_k)$ up to the relabelling $\overline{E}$ performs on the reference — which is precisely the transpose carried in the inversion below.

Third, Choi–Jamiołkowski [4, 7] inverts explicitly:

$$\widetilde{\Lambda}(\sigma) = 2^k \, \mathrm{Tr}_{\mathrm{Ref}}\left[(\sigma^T \otimes I)\, J_\Pi(\Lambda)\right].$$

where the transpose on the reference is the convention the construction applies, with $Y^T = -Y$. Equal Choi states therefore give $\widetilde{\Lambda} = \widetilde{\Lambda}'$, and by the first step $\Lambda = \Lambda'$ on every state supported in $\mathrm{supp}(\tau_X)$.

Equality of the induced logical channels determines Λ only on the image of E, which is $\mathrm{supp}(\tau_X)$ — precisely the domain the theorem asserts. □

The implementation compares canonical signatures. An equivalent double-inclusion test exists in the same module but has no caller; Lemma 1 makes the two equivalent, and the signature form is retained because it is a normal form — one comparison rather than $2 \cdot |generators|$ membership solves.

3.4 Gauge invariance

Proposition 1. The verdict is invariant under any transformation acting only on the discarded environment.

Proof. J_Π is obtained by partial trace over the environment, and the trace is invariant under isometries acting on the traced factor alone. □

Consequently two circuits differing only in ancilla convention or Stinespring dilation receive the same verdict — a property required for accepting output from compilers that do not share the reference implementation's conventions. Section 6.2 reports a family of artifacts that exercises this restriction rather than assuming it.

4 Scope

The class we treat is narrow, and the boundary is a design choice rather than an artifact of the implementation. We state it before reporting results so that every subsequent claim is read within it.

4.1 What the procedure covers

Stabilizer channels, as defined in Section 2. Within this class the decision of Section 3 is exact.

Gottesman–Knill is what makes this possible, and the same fact bounds the result: nothing here transfers to non-Clifford gates, and the procedure says nothing about circuits containing T gates or arbitrary rotations.

4.2 What is out of scope by construction

Noise, calibration, real-time decoding, and scheduling. These are physical or temporal properties; a stabilizer Choi state does not express them. Verifying that a circuit implements the intended channel is separable from verifying that a device executes it faithfully, and we address only the former.

4.3 Measurement and feed-forward

The recovery circuits verified here are coherent isometric dilations: they contain no mid-circuit measurement, and consequently no branching on measurement outcomes.

The class **admits** a symbolic treatment of the general case: a Pauli correction that is an $\mathbb{F}_2$-linear function of a syndrome is a linear map, which a stabilizer formalism can verify once rather than per branch. The current implementation **contains no syndrome handling at all** — its recovery circuits are measurement-free, so no branch arises and no such matrix is constructed. We **claim nothing** about that path and report no results for it.

Readers familiar with the Yoshida–Kitaev decoder [11] should note that its canonical form is measurement-based; the circuits here are its coherent counterpart.

4.4 What is verified and what is reported

The verdict covers semantic equality of the reduced channel on the specified subspace, conformance to the coupling map — every two-qubit gate is checked against an edge of the device graph — and invariance under change of Stinespring dilation.

Depth and two-qubit gate counts are recomputed by the verifier from the final gate list and **enter the verdict**. SWAP attribution — how many SWAPs serve movement versus restoration — is not reconstructible from the artifact format and participates in no verdict; the tables of Section 6 report it as a measurement.

4.5 Bounds on the cost measurements

The scaling results of Section 7 are measured on one family of instances: a chain architecture with random stabilizer scramblers, a single seed, and two scrambling depths. Two consequences.

First, the exponent we measure is an exponent of that family. Because the family's generator density drifts with n, it is not a degree at fixed density; Section 7.6 separates the two.

Second, no extrapolation to surface-code circuits is warranted. The densities measured range from **0.149 to 0.578**. For a surface code the same metric is derived rather than measured: a code

of distance Δ has Δ^2 data qubits and stabilizer generators of weight at most four, so mean weight $\overline{w}$ gives density $\overline{w}/\Delta^2$, and $\Delta = 5$ sits at $3.33/25 \approx 0.133$ — at the edge of the measured range rather than beyond it. The order-of-magnitude gap appears only at $\Delta \geq 13$, where constant-weight generators over a growing lattice drive the density as $1/n$. Cost projections beyond the measured range are upper bounds on a family chosen to be unfavourable, not predictions.

4.6 Two adversarial campaigns

We report two campaigns with different objects, and the figures are not interchangeable. `orelia.verifier-adversa` targets the verifier as a whole; `orelia.channel-certified-adversarial-campaign/v1` targets the channel-certified policy specifically. Section 6.2 states which is which; a reader encountering either figure in a published CSV can identify it by these format identifiers. Conflating them would suggest an inconsistency where there is none.

5 Implementation

StabCert implements the procedure of Section 3. It is a Python package with two runtime dependencies, NumPy and Stim [5], the latter providing the tableau engine on which canonical comparison rests. It is available under Apache 2.0.

The package also contains a routing procedure. It exists because the `reproducible-route` policy below requires a reference route to compare against; it is not offered as a compiler and no claim is made about its output relative to other routers.

5.1 Two policies

The tool exposes two verification policies, and the distinction is the point of the design.

`reproducible-route` requires bit-for-bit equality with the reference route. It answers *did this run reproduce the reference implementation?* and is used for regression testing.

`channel-certified` reconstructs the target from the channel specification and accepts any synthesis, routing, or Stinespring dilation whose canonical signed signature matches. It answers the question of Section 1: *does this artifact, however produced, realise the specified channel on the specified subspace?*

Circuits routed by third-party compilers fail the first policy and pass the second. That is the intended behaviour, and Section 6 reports it as such.

5.2 The certified closure

The verification path is a closed import closure, computed by AST traversal from the three entry points — compiler, verifier, command line — and enforced by the test suite. The closure comprises eleven modules; the verifier alone reaches eight. No module within it constructs a dense matrix or enumerates basis states.

The distribution also ships exploratory and instance-construction code outside this closure, including a dense stabilizer-group enumeration. No verification path can reach it, and the closure test fails if that changes. We state this because the claim of Section 3 — polynomial time, no enumeration — is a claim about the closure, not about every file in the package; a reader who greps the installed package will find the enumeration and should know why it is harmless.

The closure assertion is bidirectional: it fails both if an unreachable module becomes reachable and if a declared module is no longer reached, so the declaration cannot decay into a wish list.

5.3 What the verdict aggregates

A single invocation checks, and reports separately:

- Semantic equality of the reduced channel on the code subspace, by canonical signature comparison (Section 3).
- Coupling-map conformance: every two-qubit gate in the routed circuit is checked against an edge of the device graph, gate by gate, on the final gate list. No routing trace is required, since the final gate list is what the device executes.
- Metadata consistency: the artifact's claimed support is compared against the reconstructed Π, never used to build it (Hypothesis 1).

These are independent conditions, not components of a score: a failure in any one produces a failing verdict, and no success elsewhere compensates for it.

Resource figures are recorded alongside the verdict; Section 4.4 states which of them enter it.

5.4 Reproducibility

Published test counts are not written by hand. A synchronisation script replays the suite, parses the summary, and rewrites the counted spans in the documentation; it refuses to touch the documentation if the suite is not green, and a `-check` mode exits non-zero on divergence for use in CI.

The script guarantees that the documentation matches the local suite. It does not, by itself, guarantee anything about what a reader obtains from a checkout. That is established separately: a continuous-integration job runs the suite from a checkout on each push, under Python 3.10 and 3.12, with core dependencies only and then with every optional backend. The suite passes: 124 tests with all backends, 104 passing and 2 modules skipped without them.

6 Evaluation

6.1 Third-party routes

We compile reference recovery circuits for three fixtures and route them with two external compilers, Qiskit SABRE and pytket. Six routed artifacts result. For each, the verdict is identical:

check	result
route differs from reference	yes
`reproducible-route`	rejected
`channel-certified`	accepted
phase mutation	rejected
falsified final permutation	rejected
deterministic replay	byte-identical artifact

The first three lines are the intended behaviour of the two policies: a route produced elsewhere is not the reference route, and is nonetheless the specified channel. The last three establish that acceptance is not vacuous — a single sign flip in the tableau, or a permutation claimed but not realised, is caught.

Reproducibility. The routed artifacts reported here were produced with Qiskit 2.5.1 (SABRE [8], *decay* heuristic, seed 20260803, one trial) and pytket 2.18.1 [10] (`RoutingPass` with `LexiLabellingMethod` and `LexiRouteRoutingMethod`), on the fixtures published under `tests/fixtures/recovery_v1/`. SABRE is a stochastic search: a different seed or version yields a different route, which `channel-certified` accepts and `reproducible-route` rejects — the verdicts of this section are stable under that variation, the resource figures are not.

6.2 Adversarial campaigns

Two campaigns, with distinct objects and non-interchangeable figures.

campaign	invalid	valid	false accepts	false rejects
`orelia.verifier-adversarial-campaign/v1`	10 000	1 000	0	0
`orelia.channel-certified-adversarial-campaign/v1`	1 300	800	0	0

One family in the second campaign deserves mention, because it is the only one that separates the two possible specifications. Its members prefix the circuit by an element of the stabilizer group of τ_X: every state of the code subspace is a $+1$ eigenvector, so the action there is unchanged while the total unitary differs. These artifacts must be accepted under the subspace-restricted specification of Section 3 and would be rejected under a total-channel specification. **All 100 are accepted.** Without this family, no measurement in either campaign would distinguish the two readings of the theorem.

Mutation testing is reported by family rather than in aggregate: 10 000 sign flips is one class tested 10 000 times, not 10 000 classes.

6.3 Resources

Depth and two-qubit gate counts are recomputed by the verifier from the final gate list and enter the verdict; they are certified quantities. The `reference` column is the route the verifier reconstructs and against which difference is established — it is a baseline, not a scored competitor.

A	arch	logical	reference	SABRE	pytket
1	chain	12 / 14	46 / 62	49 / 68	34 / 41
8	chain	82 / 111	391 / 687	584 / 939	401 / 694
12	grid_2d	154 / 229	464 / 751	668 / 1177	454 / 808

Routing overhead, relative to the logical circuit:

A	arch	reference	SABRE	pytket
1	chain	3.83 / 4.43	4.08 / 4.86	2.83 / 2.93
8	chain	4.77 / 6.19	7.12 / 8.46	4.89 / 6.25
12	grid_2d	3.01 / 3.28	4.34 / 5.14	2.95 / 3.53

Both tables are to be read across, not down. Overhead rises from $A = 1$ to $A = 8$ and falls at $A = 12$; the fall is a topology effect, not a size effect — the grid offers better connectivity than the chain. Rows compare routers at a fixed instance; they compare nothing to each other.

On $A = 12$, pytket produces a circuit shallower than the reference by 10 layers while using 57 more CNOTs. Which of the two is preferable has no answer independent of a stated cost function — and this is a fact about routing, not a comparison of tools.

Between the two external routers, pytket produces both the shallower and the cheaper circuit on all three fixtures. Three instances, two topologies, one SABRE seed and one configuration each: this is what was observed, not a ranking.

6.4 Time

measurement	value
SABRE regression, 3 fixtures, end to end	50.06 s, peak RSS 88.95 MiB
pytket regression, 3 fixtures	49.67 s, peak RSS 55.97 MiB
verification alone, 2 100 campaign cases	median 77.3 ms, mean 83.9 ms, max 172.1 ms
verification at $n = 9/20/40$	0.24 s / 10.43 s / 272.23 s

The last row cross-checks Section 7.5 without having been fitted to it: doubling n from 20 to 40 multiplies time by **26.1**, against **27.5** predicted by the exponent of 4.78 measured on the upper-half window over $n = 9 \ldots 40$.

7 Cost

Two different questions are answered here, and conflating them is the way this section is most likely to be misread. **What does verification cost as instances grow?** is a question about a family. **What is the degree at fixed generator density?** is a question about the algorithm. The family we measure has a density that drifts with n, so the two answers differ. Both are stated; neither is the other. A proved bound is given in 7.4, and the measured exponents sit above it over the range measured, approaching it from above.

7.1 Counters, not seconds

Cost is reported as GF(2) operation counts, instrumented in the elimination routines. Three counters matter: the number of affine systems solved, the number of row XORs, and the number of scalar bit XORs. Row XORs are the machine-relevant measure, since NumPy vectorises a row operation; scalar bit XORs are a machine-model-independent upper bound that overcounts vectorised work by about n.

Counters are deterministic: they do not vary with machine load, scheduling, or repetition. Section 7.5 explains why wall-clock time, which does vary, is reported but not used as the complexity.

7.2 An exact identity

Over 32 instances, $n = 9$ to 40, the number of affine systems solved is not approximately but **exactly**

$$N_{\text{sys}}(n) = 28n^2 - 232n + 598.$$

with zero residual. Second differences are constant at 56; the reader can recompute this from the published CSV. This fixes the call structure of the nested elimination: the two nested loops compose to n^2 solves, not n^3.

7.3 A prediction, registered before measurement

Composing the nesting with the cost of one elimination gives the targets. That second factor was assumed on a first pass and the assumption mixed units: it took n^3, the bit count of an elimination, for its row count. Measuring one elimination against system size settles it — exponents 1.98 for row operations and 2.96 for bit operations over sizes 16 to 128 (`results/elimination_cost.json`). So $\Theta(n^2)$ eliminations, each $O(n^2)$ row operations and $O(n^3)$ bit operations, predict degrees 2, **4** and **5**.

Three lines of measurement decided what 32 swept points had not, because the factor had never been measured — only supposed.

Observed exponents cannot be compared to those targets directly: they are inflated by lower-order terms. The counter whose degree is known exactly calibrates that inflation on the same instances. The inflation is derivable in closed form from the identity above — the tangent exponent is $n \cdot N'_{\text{sys}}(n)/N_{\text{sys}}(n)$, so the bias is

$$\beta(n) = \frac{232n - 1196}{28n^2 - 232n + 598} \longrightarrow \frac{232}{28} = \frac{8.286}{n}.$$

matching the measured secant bias to within 1.2×10^{-3} across the range.

With $\tau = max(observed(S) - 2, 0.1)$ and $excess = observed - target$, the rule registered before the sweep was: **confirmed** if $excess \leq \tau$ and the exponent is still falling; **rejected** if $excess > 2\tau$ or the exponent rises; **undecided** otherwise, which alone justifies extending the range. The floor of 0.1 prevents the test from degenerating as the bias vanishes; it did not bind here, the measured bias being 0.2896.

Registering this mattered. At $n \leq 30$ the rule returned *undecided* for `row_xors` by 0.007 — an excess of 0.3914 against a tolerance of 0.3843. A threshold chosen after seeing that number would have been chosen differently.

7.4 Result

Windows are the lower and upper halves of $n = 9 \ldots 40$, 16 points each.

counter	target	lower	upper	trend	excess	tolerance	verdict
`affine_systems_solved`	2	2.690	2.290	-0.400	—	—	calibrator
`row_xors`	4	6.285	**4.907**	-1.378	+0.907	0.290	**rejected**
`scalar_bit_xors`	5	7.643	**6.106**	-1.536	+1.106	0.290	**rejected**
`verify_seconds`	—	4.711	4.780	**+0.068**	—	—	not a verdict

Both are rejected at the registered threshold, and the earlier reading of this table said the opposite. It compared the same measurements against targets of 5 and 6, found excesses of -0.09 and +0.11, and recorded a confirmation. Those targets came from the unit-mixed composition; a pre-registered threshold can only validate the prediction it is given, and it confirmed a wrong one with every appearance of rigour. The threshold did not fail. The hypothesis did.

Rejection here does not mean the cost is worse than claimed. The bound below is tighter than what the section previously asserted, and the measurements are unchanged. It means the measured exponents have not reached their asymptote over $n = 9 \ldots 40$: both are falling steeply, by 1.38 and 1.54 across the range, which is the signature of a function approaching its degree **from above** — consistent with an n^4 and an n^5 asymptote, and not with 5 and 6.

Single-step exponents are not reported — at the top of the range they are unstable enough to be meaningless.

Proposition 2 (cost bound)**.** Deciding signature equality for a specification of accessible width n performs $\Theta(n^2)$ eliminations, each on a system of $O(n)$ rows and $O(n)$ columns, hence $O(n^4)$ row operations and $O(n^5)$ bit operations over $\mathbb{F}_2$, independently of generator density.

Proof. The elimination count is the exact identity of 7.2. One elimination performs at most $O(n)$ pivots, each exclusive-or of the pivot row into at most $O(n)$ rows, hence $O(n^2)$ row operations; each row spans $O(n)$ bits, hence $O(n^3)$ bit operations. Neither factor depends on how dense the systems are — only on their dimensions, which are fixed by n. Multiplying gives the stated bounds. □

This is what makes *polynomial* in the abstract a proved claim rather than a measured one. The measurement of 7.4 is then a second, independent statement: that the bound is not grossly loose, since the exponents descend towards it rather than towards something far below.

7.5 Wall time is not the complexity

Timing gives an exponent of **4.780** on the upper-half window over $n = 9 \ldots 40$. An unrelated instance family — four structural preflights over $choi\_qubits = 26 \ldots 32$ — gives a log-log exponent of 4.55 for its construction-and-verification time. The agreement is a consistency check across harnesses, not a second measurement of the same quantity: the ranges, the parameters and the timed sections differ.

Time grows **more slowly** than the operation counts (4.780 against 4.907 for row XORs), and its exponent *rises* with n — 4.711, then 4.780 — while every counter's falls. The cause is vectorisation: NumPy performs a row XOR in roughly constant time, so measured seconds track row operations rather than bit operations, and converge on that exponent from below.

The two series therefore bracket the answer from opposite sides: counters descending from 6.285 to 4.907, time ascending from 4.711 to 4.780, meeting within 0.13. **Two independent series converging from above and below constrain more than a two-parameter fit to either one.**

7.6 Density, and what the exponent is an exponent of

The measured family's generator density is not constant: it drifts as $n^{-0.238}$. This is mechanical rather than incidental — at fixed scrambler depth the number of two-qubit gates grows linearly while the number of matrix entries grows quadratically, so normalised density must fall.

A paired design was registered to neutralise density by holding scrambler depth fixed. It failed on its own terms: the density contrast between the two depths itself drifts from 1.259 to 2.455, an exponent of $n^{0.582}$ against a tolerance of 0.384, so the ratio exponent conflates a degree effect with a widening gap and is not readable. Density does not stay neutralised; it must be modelled.

Pooling both depths at 22 matched widths — where the elimination structure is identical, the same $N_{\mathrm{sys}}(n)$ at every width — gives

$$\log(\mathrm{row\_xors}) = a + b \log n + c \log(\mathrm{density}), \qquad b = 6.313, \quad c = 1.410.$$

Before reading those coefficients, two gates. The model must reproduce each arm's observed exponent: it does, to ± 0.077 against a tolerance of 0.384 (dense, observed 6.054 against 5.977; sparse, observed 5.079 against 5.156). And the same model fitted to the exactly-known counter,

whose true density elasticity is zero, must return approximately zero: it returns $c_0 = 0.006$. The method invents essentially nothing.

$c = 1.410$ against a threshold of 0.384 and a false-positive floor of 0.006: **density governs a real share of the cost.** $b = 6.313$ is an all-points fit and carries the same finite-size inflation the calibrator shows on that fit ($b_0 = 2.611$ against a true 2, so $+0.611$), giving a bias-corrected degree at fixed density of about **5.7** — higher than the family exponent of 5, as it must be, since density falls with n and cost rises with density. The relation that reconciles the two is $family exponent = b + c \cdot \delta$, with δ the density drift of the arm.

One weakness to report: the maximum log residual is 0.408 for `row_xors` against 0.104 for the calibrator. The model satisfies the exponent gate but fits appreciably less well pointwise; $c = 1.41$ has the right sign and order of magnitude, and its second decimal does not.

This is why Section 4.5 forbids extrapolation to a surface code. The relevant comparison is of drift regimes: the dense arm drifts as $n^{-0.238}$, the sparse arm as $n^{-0.820}$, a surface code as n^{-1} — constant-weight generators over a growing lattice. The sparse arm already approaches the surface-code density regime without having its locality. What is captured is density; what is not is locality.

7.7 One method tried and rejected

Fitting $e(n) = \gamma + K/n$ to the sequence of local exponents extrapolates the asymptotic degree directly and would have terminated the range question cleanly. Run first against the calibrator, whose degree is exactly 2, it returns $\gamma = 1.941$ with a jackknife spread of $[1.939, 1.942]$ over $n = 9 \ldots 40$ — excluding the truth inside an interval some thirty times too narrow.

The $1/n$ form is only the leading correction, so its residuals are systematic, and a jackknife measures the stability of a fit rather than its accuracy. The method is not used. We record it because the failure is not specific to these data: any resampling estimate of uncertainty will tighten around the wrong value when the error is systematic, and only a quantity with a known answer reveals it.

8 Related work

Three lines of work address adjacent problems. We have not executed any of the tools discussed below; the characterisations that follow are drawn from their published documentation and papers.

Compiler certification. Proving the transformation itself, once and for all, is stronger where it applies — and it applies only to compilers whose source is available and under the verifier's control. SABRE and pytket are neither. Translation validation, certifying each output rather than the producer, is the classical answer to that situation, and it is the one we take.

Equivalence checking. QCEC [2] compares a compiled circuit against a reference circuit. Its decision-diagram engines provide exact equivalence checking by comparing canonical representations of the implemented operators, with support for ancilla and garbage qubits and a notion of partial equivalence defined on measurement distributions, although resource consumption can be exponential in the worst case; a ZX-calculus engine [9] is often effective at establishing equivalence but, being based on an incomplete rewriting procedure, cannot in general establish inequivalence. The two engines are combined deliberately, and exploiting the $GG'^{\dagger}$ structure is the published contribution that keeps the diagrams small in practice.

Like our approach, QCEC relies on canonical representations. The distinction is not canonicity but the object represented and the input required: QCEC canonically represents operators and compares two of them, whereas we canonically represent a stabilizer channel restricted to a code

subspace and compare one against a target reconstructed from a channel specification. This is not a deficiency of QCEC — in its own use case, checking that a compiler preserved the source circuit, the reference *is* the specification and the question is well posed. The gap matters only where no reference exists yet.

Deductive verification. Qbricks [3] verifies circuit-building quantum programs, reaching parametric implementations of Shor's order finding, quantum phase estimation and Grover's search, with high proof automation over parametrized path sums in Why3. Its scope is far wider than ours in the class of circuits reached; its documented workflow is to write a program in its own DSL, and we found no documented import path for an artifact produced elsewhere. The two tools therefore answer different questions and admit no meaningful quantitative comparison: proof effort and verification time are not the same quantity, and tabulating them side by side would suggest a commensurability that does not exist.

Summary. The axis that separates this work from all three is the input. Compiler certification takes a compiler; equivalence checking takes two circuits; deductive verification takes a program. We take an artifact of unknown provenance and a channel specification.

References

[1] Scott Aaronson and Daniel Gottesman. Improved simulation of stabilizer circuits. *Physical Review A*, 70(5):052328, 2004.

[2] Lukas Burgholzer and Robert Wille. Advanced equivalence checking for quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2021.

[3] Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. An automated deductive verification framework for circuit-building quantum programs. In *ESOP*, pages 148–177, 2021.

[4] Man-Duen Choi. Completely positive linear maps on complex matrices. *Linear Algebra and its Applications*, 10(3):285–290, 1975.

[5] Craig Gidney. Stim: A fast stabilizer circuit simulator. *Quantum*, 5:497, 2021.

[6] Daniel Eric Gottesman. *Stabilizer Codes and Quantum Error Correction*. PhD thesis, California Institute of Technology, 1997.

[7] Andrzej Jamiołkowski. Linear transformations which preserve trace and positive semidefiniteness of operators. *Reports on Mathematical Physics*, 3(4):275–278, 1972.

[8] Gushu Li, Yufei Ding, and Yuan Xie. Tackling the qubit mapping problem for NISQ-era quantum devices. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 1001–1014, 2019.

[9] Tom Peham, Lukas Burgholzer, and Robert Wille. Equivalence checking of quantum circuits with the ZX-calculus, 2022.

[10] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. t|ket⟩: A retargetable compiler for NISQ devices. *Quantum Science and Technology*, 6(1):014003, 2020.

[11] Beni Yoshida and Alexei Kitaev. Efficient decoding for the Hayden–Preskill protocol, 2017.